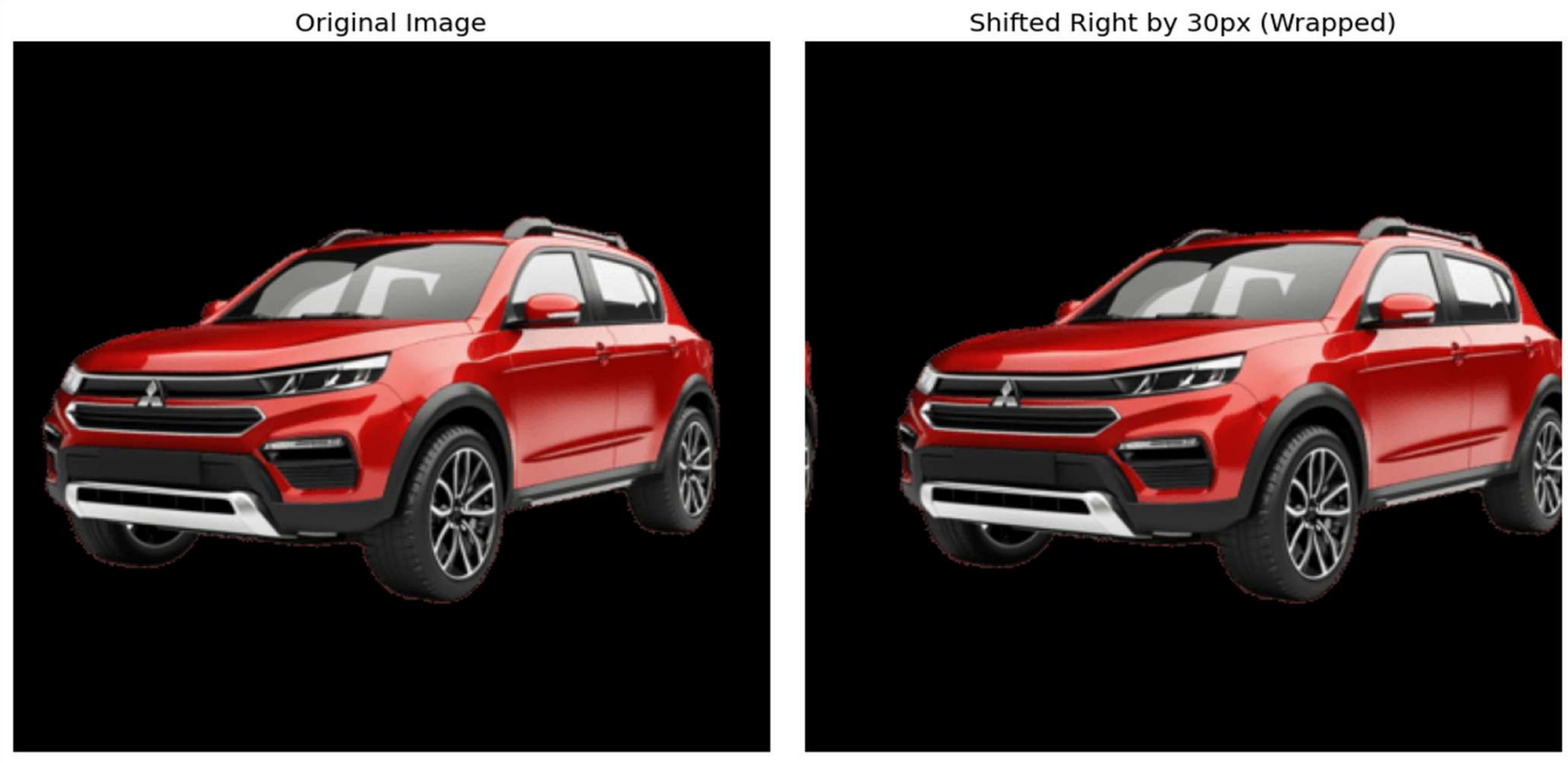
Evaluation Component	Date Of Announcement to Students	Deadline of Submission	Weightage	Comments
Quiz			25%	
End Term Exam			35%	
Projects/Kaggle/Assignments (best 2 of 3)			40%	Project Score = Test case based evaluation score * [Project understanding section in end-term exam]

Evaluation Component	Date Of Announcement to Students	Deadline of Submission	Weightage	Comments
For Internship Exempted & Course Repeat Students				
Viva			25%	
End Term Exam			35%	
Projects/Kaggle/Assignments (best 2 of 3)			40%	Project Score = Test case based evaluation score * [Project understanding section in end-term exam]

Homework: Can a CV system recognize the same car?



Homework: What does a computer see?

Imagine comparing the images pixel-by-pixel.



Interactive homework review

🧠 2-minute challenge

SHOULD STILL WORK

Find 2 transformations where a good CV system should still recognize the object.

SHOULD BECOME DIFFICULT

Find 2 transformations where recognition should become difficult.

🧠 2-minute challenge

SHOULD STILL WORK

Find 2 transformations where a good CV system should still recognize the object.

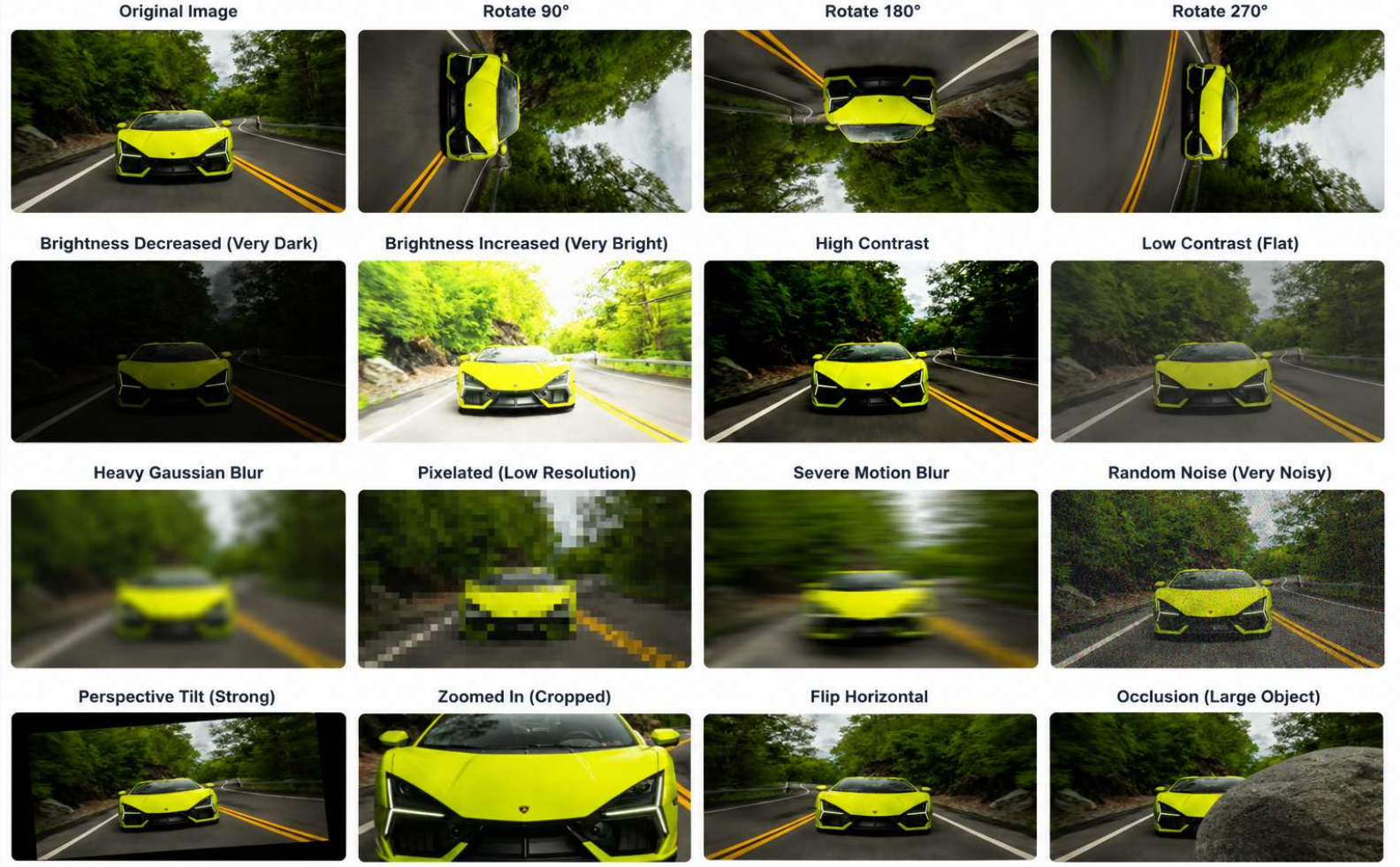
Examples: translation, small rotation, scale, lighting.

SHOULD BECOME DIFFICULT

Find 2 transformations where recognition should become difficult.

Examples: heavy occlusion, blur, extreme rotation, severe crop, severe translation.

Homework: What severe transformations does to image.



Homework: Should the model still recognize it?

↔ Move pixel position

YES / NO

↻ Rotate image

YES / NO

🔍 Make smaller

YES / NO

☀ Change brightness

YES / NO

✂ Crop part

YES / NO

↻ Rotate 180°

YES / NO

Takeaway: Yes/No-both are the possibilities based on strength of transformation

The takeaway

What did our homework actually teach us?

An object's pixels can change because of:

Translation • Rotation • Scale • Lighting • Occlusion • Viewpoint

BUT

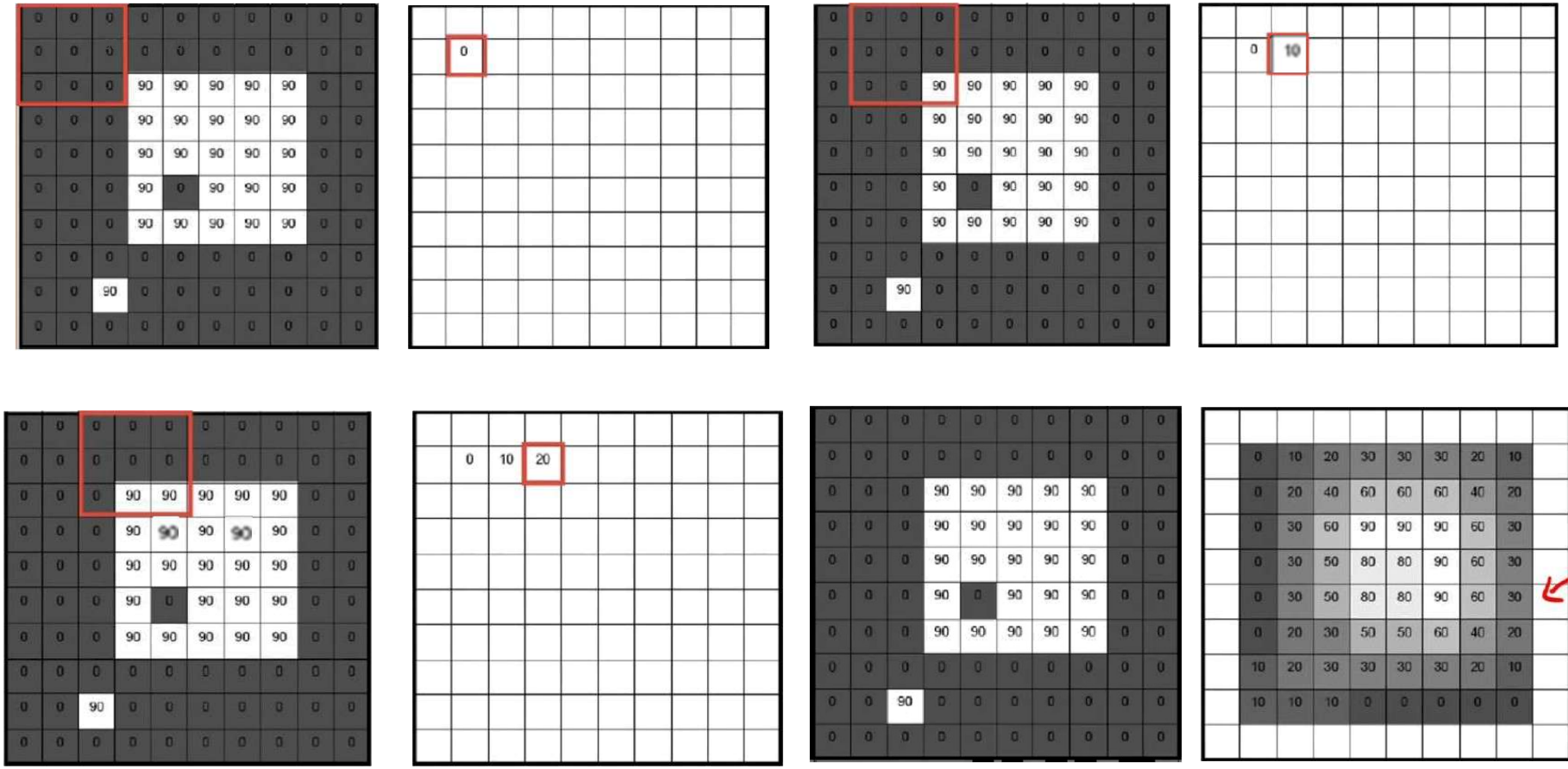
its identity may remain unchanged.

PIXEL SIMILARITY ≠ SEMANTIC SIMILARITY

Transition to today's lecture

Recap: Applying Filters

$$1/9 \cdot \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$



Recap: Derivatives with Filters ≈ Edge Filter



Gradient in x Direction

$$S_x = \frac{\partial f(x,y)}{\partial x}$$

-1	1



X-Derivative of Gaussian



Y-Derivative of Gaussian

Gradient in y Direction

$$S_y = \frac{\partial f(x,y)}{\partial y}$$

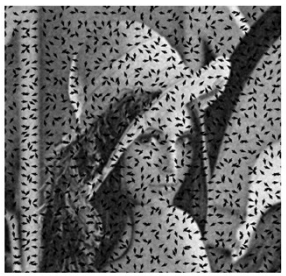
-1
1

or

1
-1



$$|\nabla S| = \sqrt[2]{S_x{}^2 + S_y{}^2}$$



$$\alpha(x,y) = \tan^{-1}\left(\frac{S_y}{S_x}\right)$$

- For 2D function, $f(x,y)$, the partial derivative is:

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \to 0} \frac{f(x+\varepsilon,y) - f(x,y)}{\varepsilon}$$

- For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1,y) - f(x,y)}{1}$$

- Gradient Magnitude

$$|\nabla S| = \sqrt[2]{S_x{}^2 + S_y{}^2} \approx |S_x| + |S_y|$$

- Gradient Orientation

$$\alpha(x,y) = \tan^{-1}\left(\frac{S_y}{S_x}\right)$$

Todays Session

SIFT/ORB → Feature Matching → RANSAC → Homography → Image Stitching

From local features to a stitched panorama

SIFT Summary

SIFT = Scale-Invariant Feature Transform

For each stable keypoint, SIFT produces:

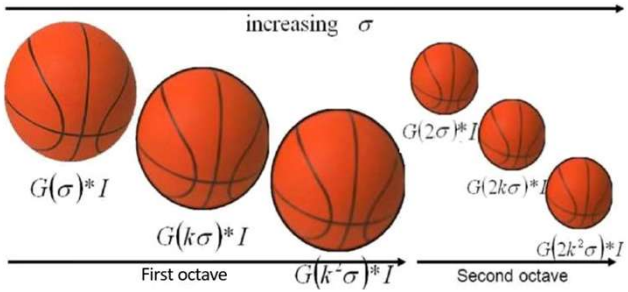
- location (x, y)
- characteristic scale
- dominant orientation(s)
- a 128-dimensional descriptor

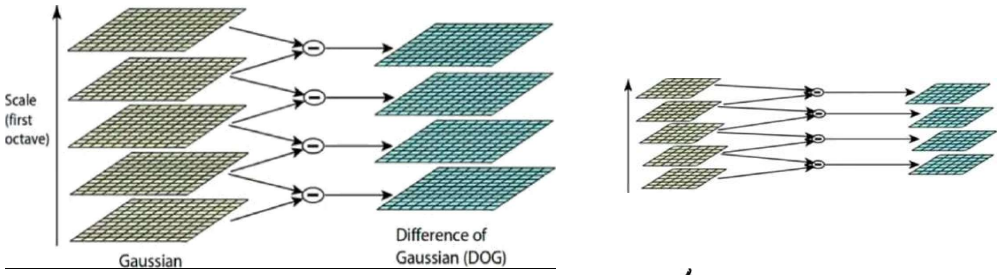
The descriptor summarizes local gradient structure rather than raw pixels.

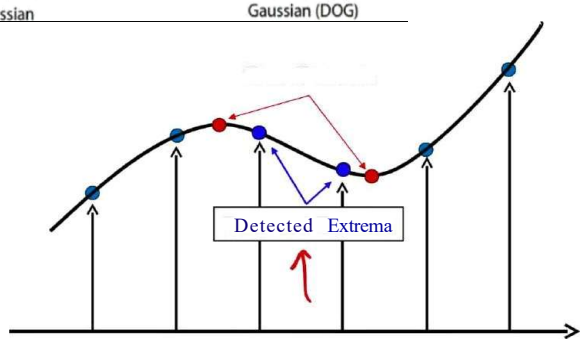
SIFT-style scale-space candidates



Recap: SIFT



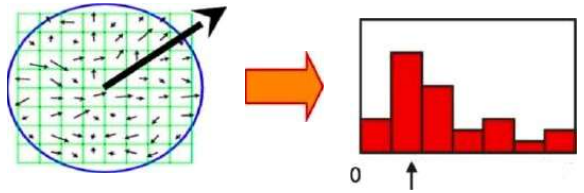




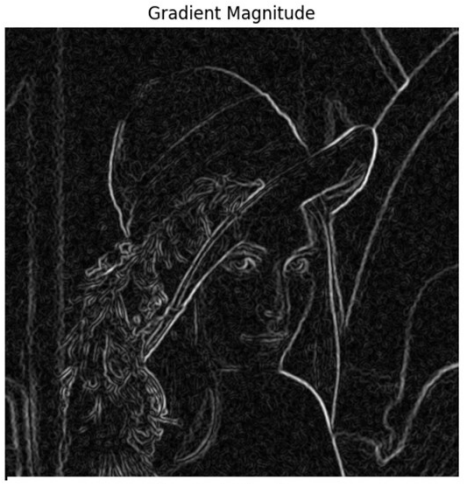
Keypoints Localization

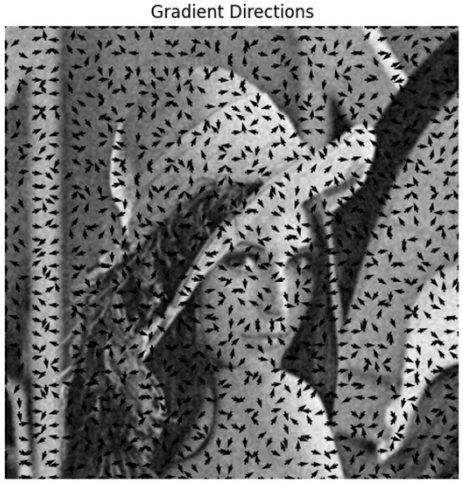
- Reject points with strong edge response in one direction only
- Edge Elimination - How?
  - Similar to Harris corner detector!

Orientation Estimation



Why gradients?







Magnitude = strength of local intensity change.

Orientation = direction of that change.

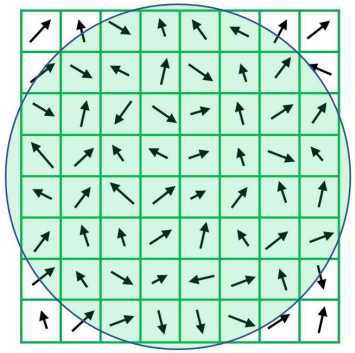
SIFT uses these gradients to describe local structure robustly.

Assign the keypoints orientation

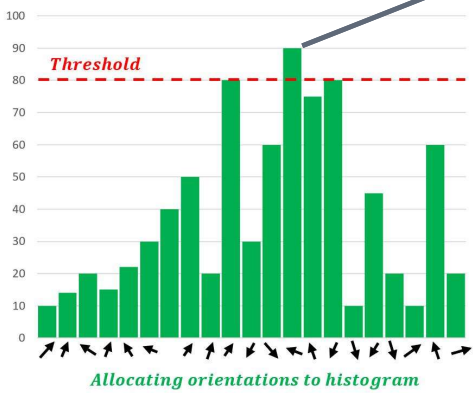
Take gradients around the keypoint and build an orientation histogram.

Highest peak = dominant orientation

≈ 36 bins (10° each)



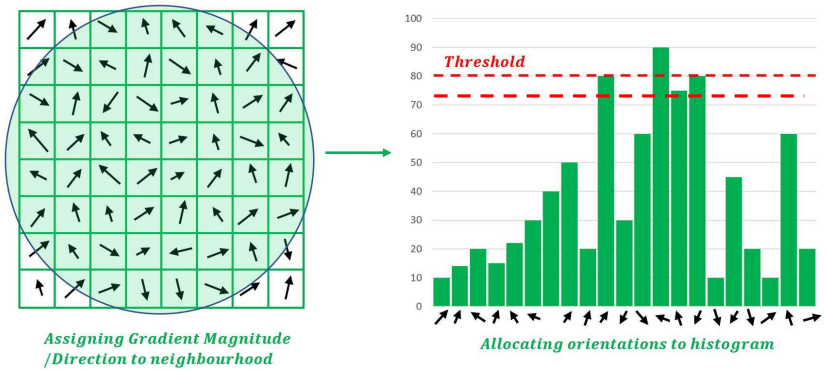
Assigning Gradient Magnitude /Direction to neighbourhood



Allocating orientations to histogram

- The histogram accumulates weighted gradient strength.
- The largest peak defines the keypoint orientation.
- This lets the descriptor use a local orientation reference → rotation robustness.

The confusing part: what if there are two strong orientations?



Two strong gradient directions

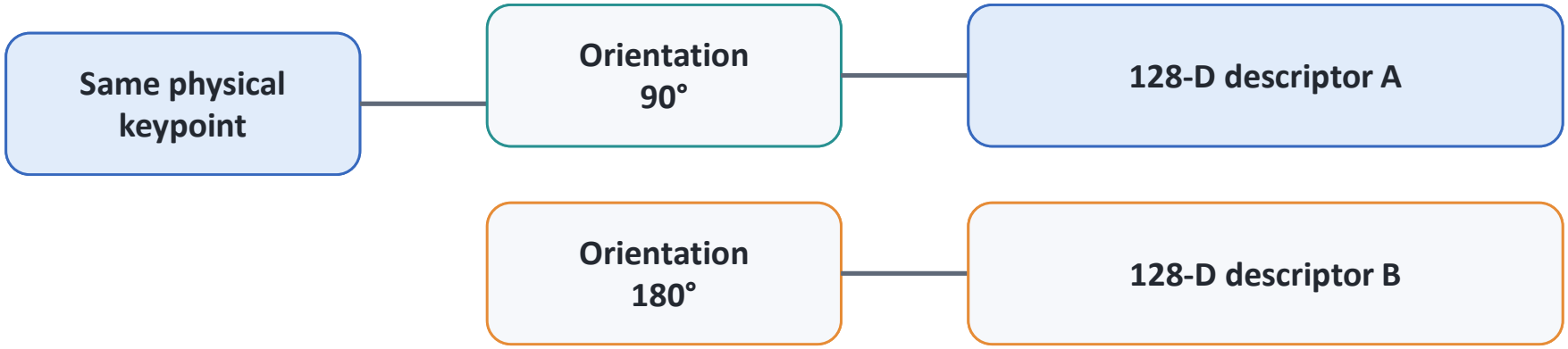
Peak A
90°

Peak B
180°

If a secondary peak is strong enough (≈ 80% of the maximum), SIFT assigns another orientation to the same location.

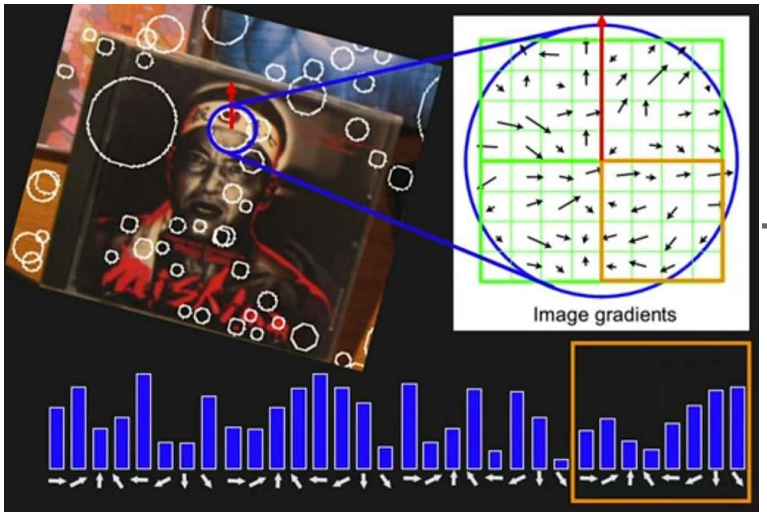
Result: same (x,y,scale), but potentially two descriptors:
Descriptor A → orientation 90°
Descriptor B → orientation 180°

Why create multiple orientations?



SIFT prefers preserving strong orientation hypotheses instead of forcing an ambiguous local structure into one arbitrary orientation.

How 16×16 becomes a 128-D descriptor

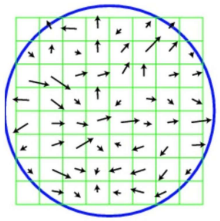


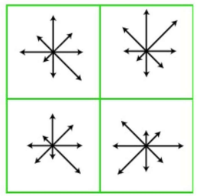
16×16 neighborhood → 4×4 spatial cells

One 4×4 cell
16 pixels

8-bin orientation histogram

Each cell produces 8 accumulated, weighted gradient-strength values.





[0.2, 0.1, 0, 0, 0.5, 0.2, 0, 0]
[0.8, 1.0, 0, 0, 0, 0, 0, 0]
[0, 0.4, 0, 0.7, 0, 0, 0.1, 0]
[0, 0, 0.1, 0.5, 0, 0, 0, 0.3]

4 × 4 × 8 = 128

Summary: Sift

1. Find stable keypoints

Where? At what scale?

2. Assign orientation

Which direction dominates?

3. Build descriptor

What does the neighborhood look like?

4. Match descriptors

Which feature corresponds?

5. Reject outliers

Do matches agree geometrically?

6. Stitch

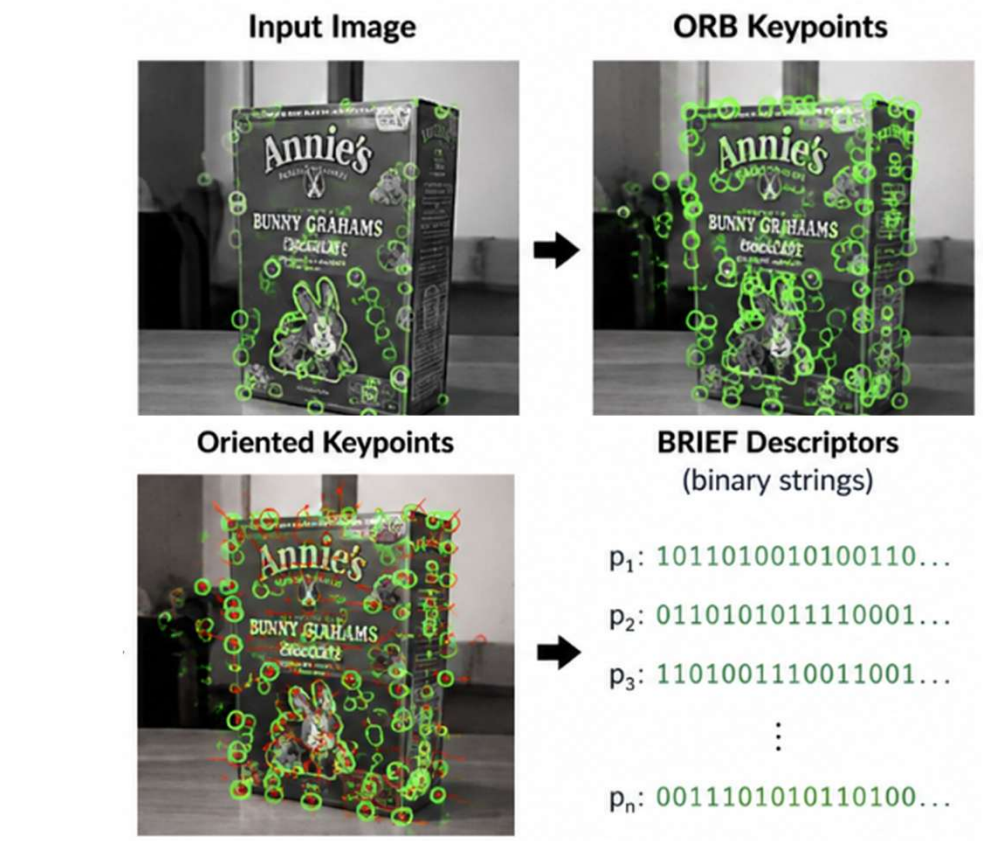
Warp + align + blend

ORB (Oriented FAST and Rotated BRIEF)

For each stable keypoint, ORB produces:

- location (x, y)
- pyramid of scales (blur first and then downsample)
- dominant orientation
- a compact binary descriptor

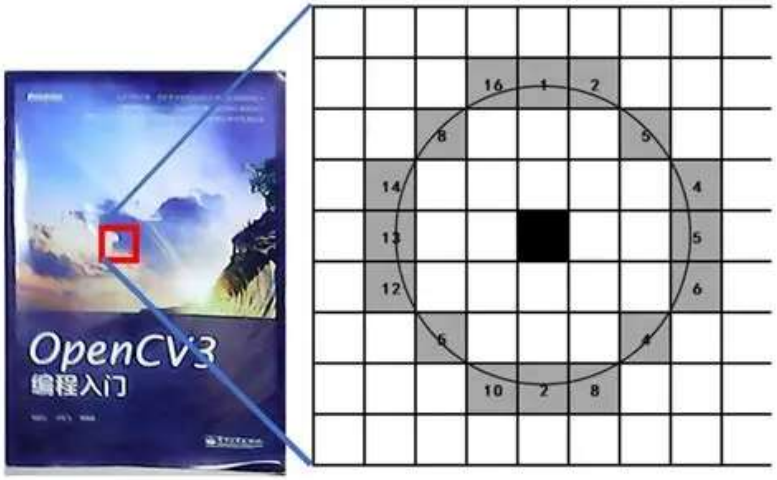
The descriptor compares pixel intensities rather than storing raw pixels or gradient histograms.



ORB Step 1 — FAST Finds Keypoints

FAST = Features from Accelerated Segment Test

- Choose a candidate pixel P.
- Inspect 16 pixels on a circle around P.
- Check whether enough consecutive pixels are much brighter or darker than P.
- If the test passes, P is a corner / keypoint candidate.



16-pixel circle

Enough consecutive brighter / darker pixels → corner

Why FAST?

Only simple intensity comparisons are needed → very fast.

ORB Step 2 — Orientation + Rotated BRIEF

FAST gives a location, but we also need the keypoint's orientation.

1. Estimate orientation

ORB uses the intensity centroid of the local patch to estimate an angle θ.

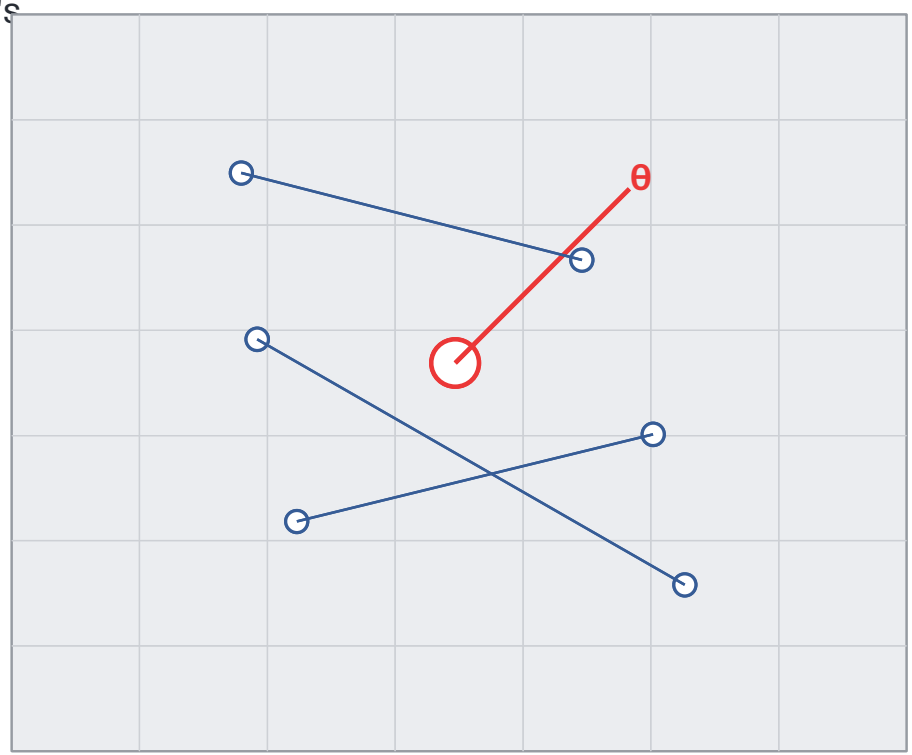
2. Build a BRIEF descriptor

Pick pairs of pixels and compare their intensities:

I(A) < I(B) → 1
I(A) ≥ I(B) → 0

Repeat many tests → binary descriptor such as

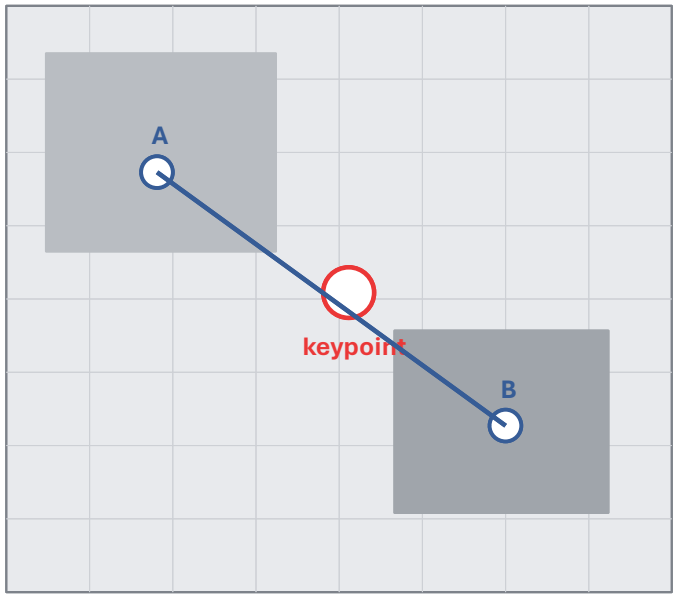
101100101011001O...



Rotate the sampling pattern by θ → rotation robustness

BRIEF — How do we describe a keypoint?

Take a small image patch around the keypoint.

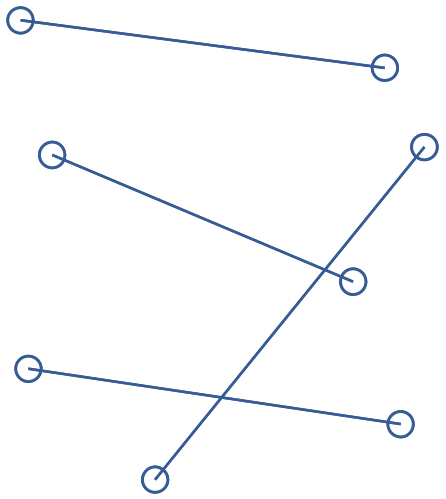


A is DARKER than B

I(A) < I(B) —— **1**

One comparison → ONE BIT

Repeat this with many pairs of pixels



1 0 1 1 0 0 1 0 …

That binary string is the BRIEF descriptor.

Why does BRIEF work?

It describes the local intensity pattern not the exact pixel values.

Same feature, different brightness

Image 1	Image 2
A = 80 B = 150	A = 110 B = 180
80 < 150 → 1	110 < 180 → 1

Absolute values changed, but the comparison stayed the same.

But fixed BRIEF is NOT rotation invariant



Rotate the image → fixed sampling points no longer correspond.

ORB rotates the BRIEF sampling pattern by θ

ORB Step 3 — Match Binary Descriptors

ORB descriptors are binary, so matching uses Hamming distance.

Example

Descriptor A

10110110

Descriptor B

10011110

Hamming distance : Count how many positions are different between two binary strings.

Hamming Distance can be efficiently calculated with XOR

XOR **00101000**

Why ORB is fast

- FAST uses simple intensity comparisons.
- BRIEF produces compact binary descriptors.
- Hamming distance can be computed with XOR + bit counting.
- Image pyramids provide practical scale robustness.

Hamming distance = count of 1's in XOR string = 2

How to stitch 2 Images?

① Input Image 1



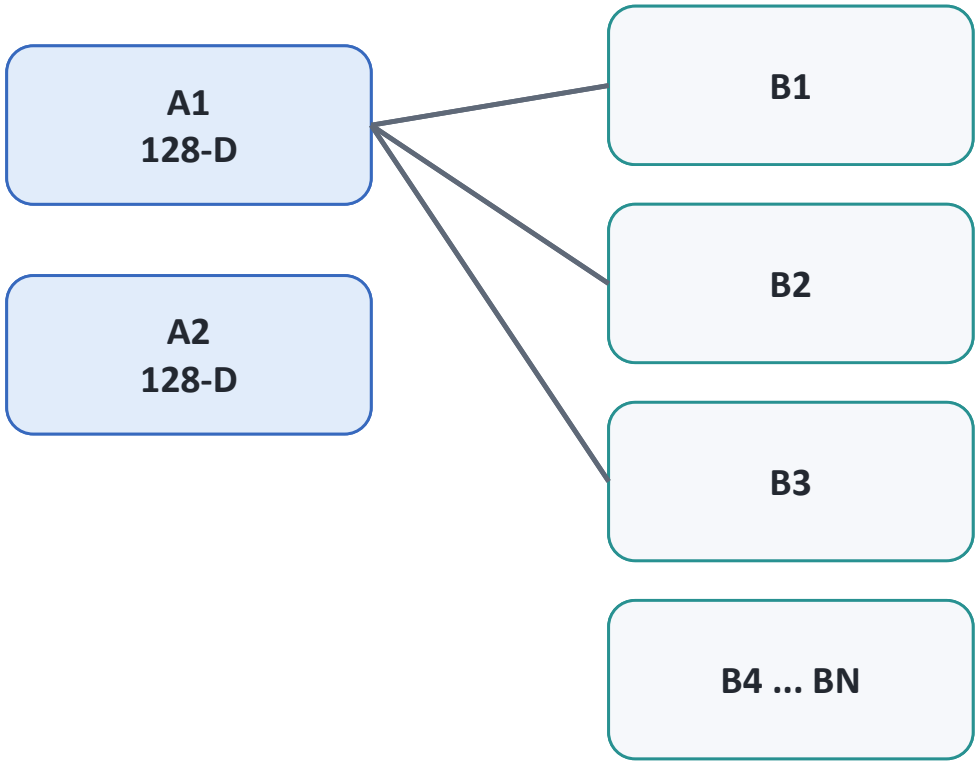
② Input Image 2



④ Final Panorama



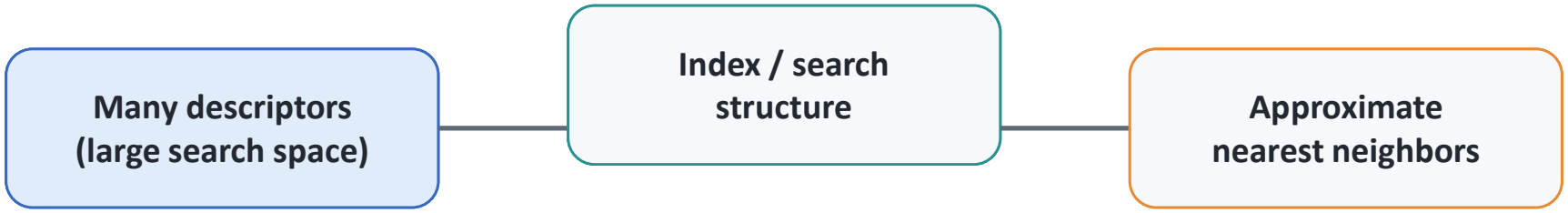
Matching descriptors — Brute Force (BF) Matcher



For each descriptor in A:
- compare against candidates in B
- compute distance (SIFT → commonly L2)
- choose nearest neighbor(s)

Simple + exact, but can be expensive when the number of descriptors is large.

FLANN — faster nearest-neighbor search



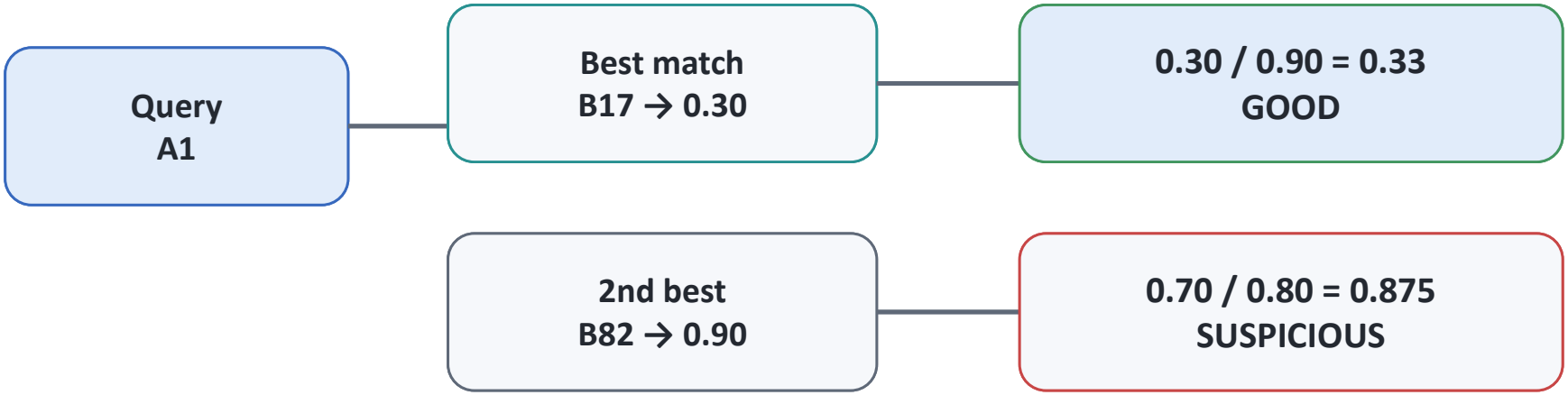
FLANN = Fast Library for Approximate Nearest Neighbors.

It avoids blindly checking every possible descriptor pair.

Trade-off: faster search, approximate rather than guaranteed exact nearest neighbor.

Rule of thumb: BF is simple; FLANN becomes attractive as the search problem grows.

Lowe's ratio test — is the match actually distinctive?



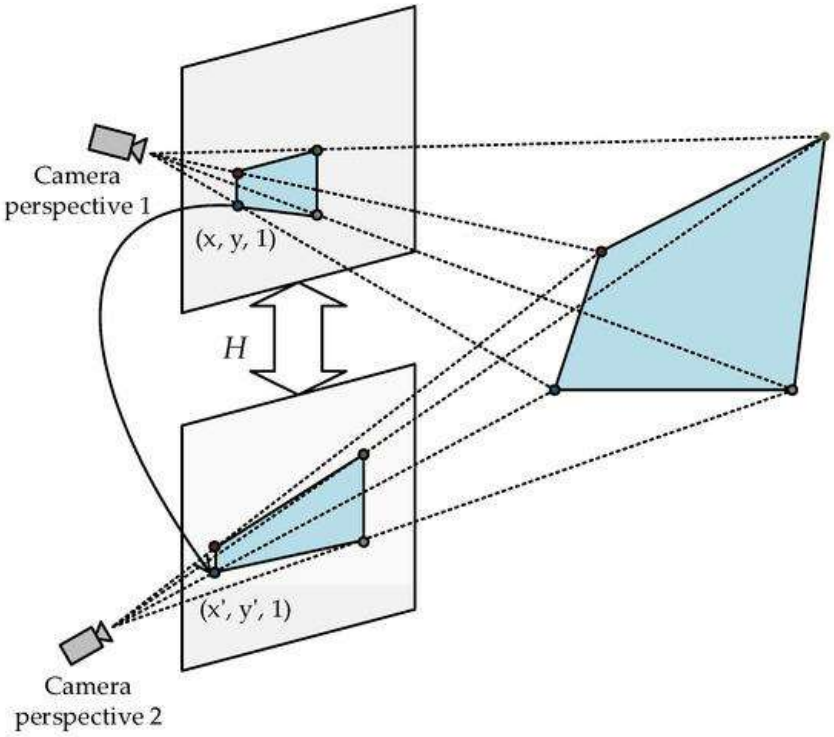
Idea: accept a match when the best candidate is substantially better than the second-best candidate. Common thresholds are around 0.7–0.8, depending on the task.

Homography — the geometric transformation

Point in Image 1
$[x, y, 1]^T$

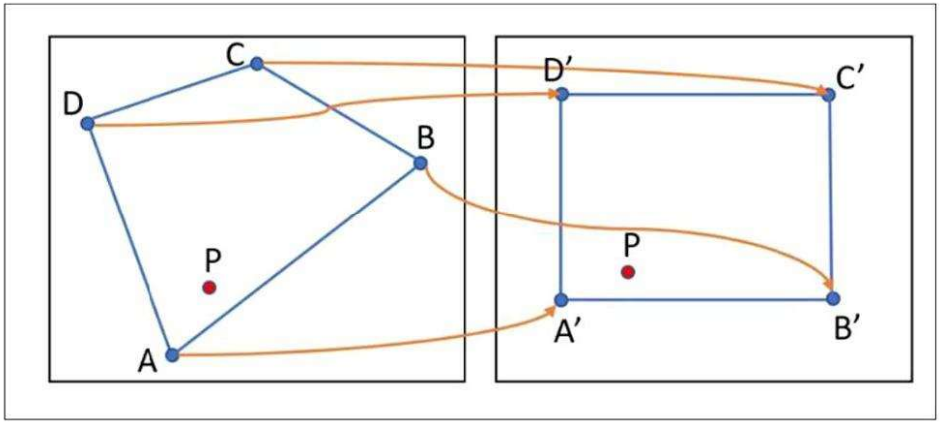
H
3×3 matrix

Point in Image 2
$[x', y', 1]^T$



$$[x'\ y'\ 1]^T \sim H\,[x\ y\ 1]^T$$

- A homography maps one image plane to another projectively.
- At least 4 point correspondences are needed to estimate H.



Why 4 points?

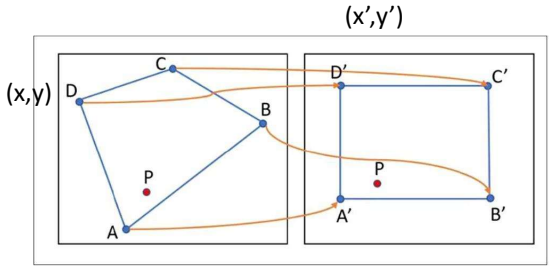
$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

$$x' \sim Hx$$

The ~ means "equal up to scale.“, this is important as now we have

$$9 - 1 = \boxed{8}$$

Independent Parameters



The homography says:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

If you expand this, you get two independent equations:

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

So:

> **ONE point correspondence gives us 2 equations.**

Therefore to solve for 8 independent variables we at least need 4 points of correspondence

Numerical

Four corresponding points are identified between two images:

Point	Image 1 (x,y)	Image 2 (x′,y′)
1	(0, 0)	(0, 0)
2	(1, 0)	(2, 0)
3	(1, 1)	(2, 2)
4	(0, 1)	(0, 2)

Assume the homography is:

$$x' \sim Hx$$

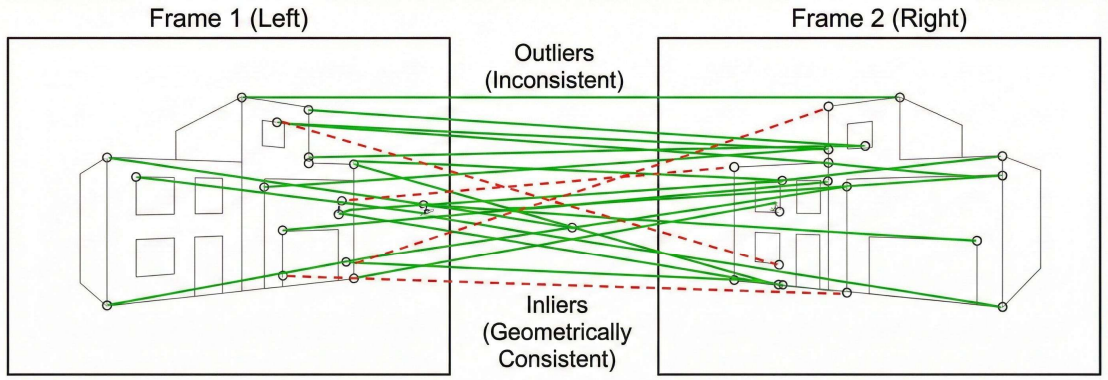
where H is

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}$$

Task: Find the homography matrix H that maps Image 1 → Image 2.

RANSAC: From Similarity Matches to Geometrically Consistent Matches

Suppose matching leaves 50 candidate correspondences: 40 correct + 10 incorrect.



Randomly sample a minimal set

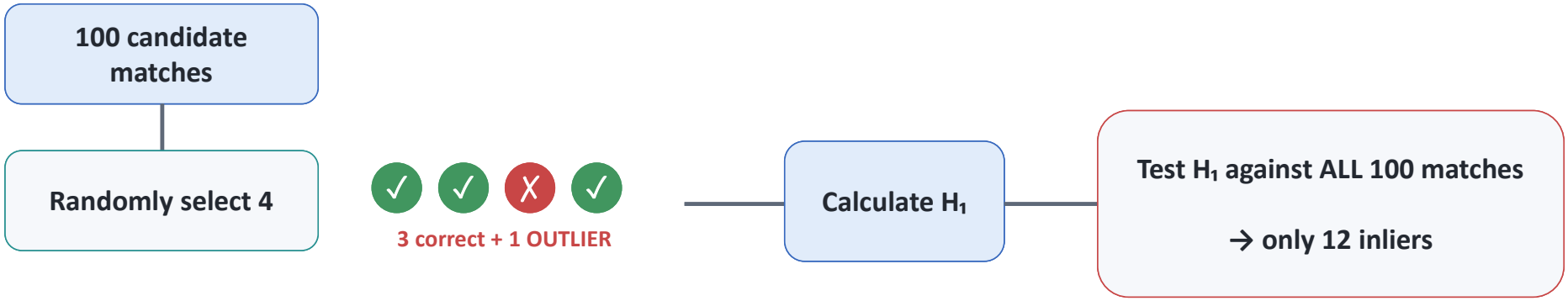
4 matches
→ estimate H

Test all matches
→ count inliers

Good model: many matches have small reprojection error. Bad model: few matches agree. Repeat → keep the model with the strongest consensus.

RANSAC: What if the 4 selected points contain an outlier?

A homography needs 4 correspondences — but the random sample may itself contain outliers.

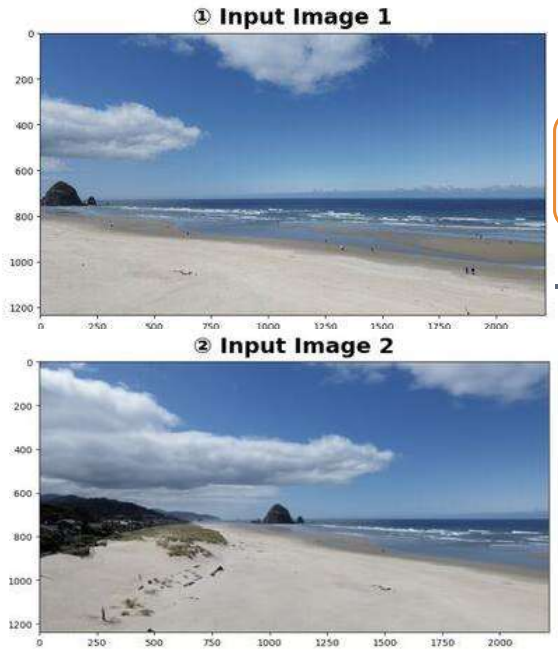


A bad 4-point sample produces a bad H — but RANSAC simply rejects that hypothesis and tries another.



Panorama stitching — the complete process

Image 1 + Image 2 → SIFT / ORB → BF / FLANN → Ratio test → RANSAC → Homography H → Warp Image 2 → Blend



H 3×3 matrix



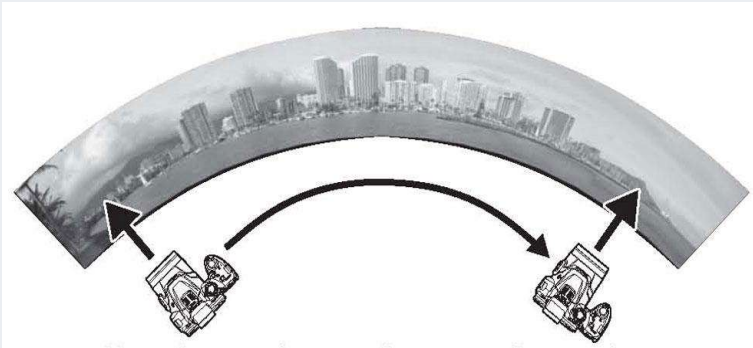


When does homography work well?

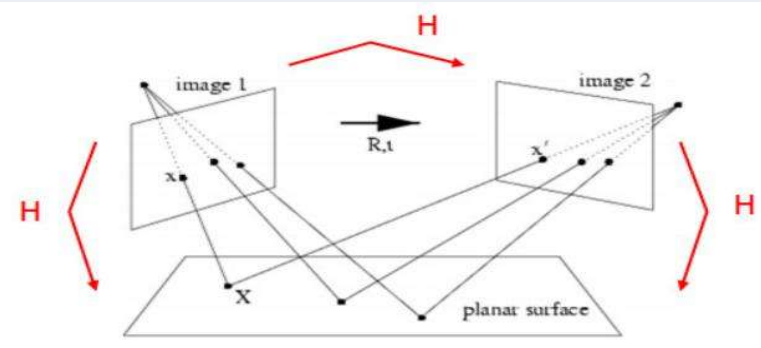
Panorama
camera rotation

Planar scene
(document / wall)

Small depth variation
≈ okay



Camera arc over a landscape



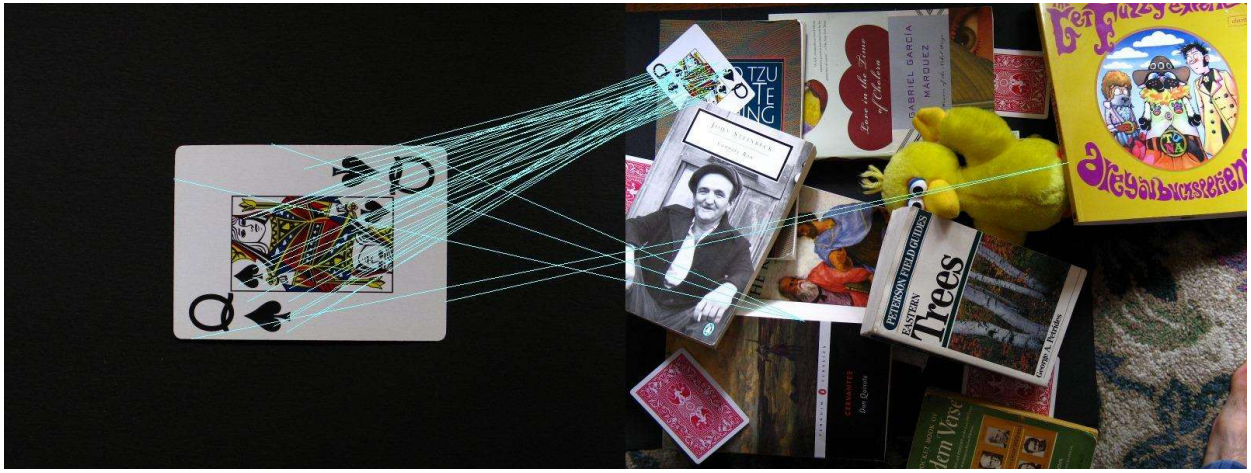
Photographed poster with perspective distortion

Key idea: a single homography assumes one coherent projective relationship between the image planes.

Where SIFT is great

SIFT/ORB

- Rotation robustness
- Scale changes
- Moderate viewpoint changes
- Distinctive local texture
- Partial overlap
- Local feature matching
- SLAM / panorama building
- No training data required



Where they can still fail

Textureless regions

Blank wall / sky → few distinctive keypoints

Repetitive patterns

Identical windows / tiles → ambiguous matches

Large viewpoint change

Local appearance changes dramatically

Extreme illumination

Gradient structure can change substantially

Motion blur

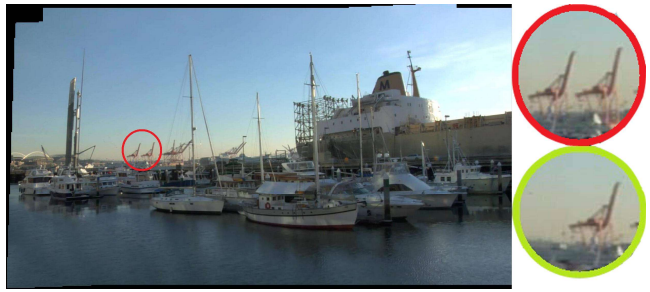
Weak / smeared gradients

Dynamic objects

Moving cars/people violate static-scene assumptions

Parallax

One homography cannot align different depths perfectly



The parallax problem — the classic panorama failure



One H?

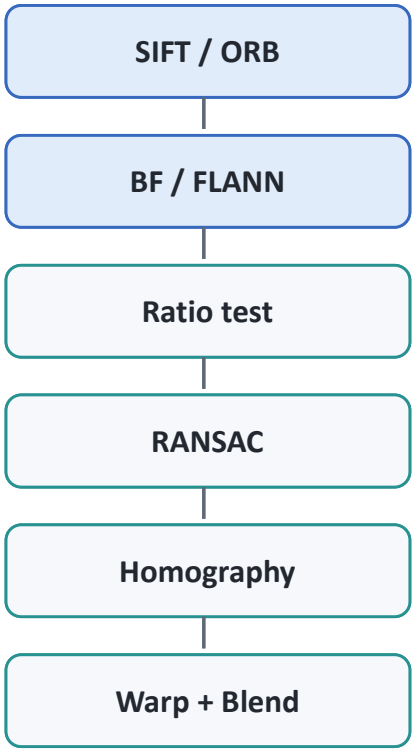
Result: ghosting / seams

Foreground needs one transformation

Background needs another transformation

If the camera translates and the scene has depth, a single homography may not be enough.

The final mental model



“What does this local patch look like?”

“Which patch in the other image looks similar?”

“Is that match clearly better than alternatives?”

“Do these matches agree with one geometry?”

“What is that geometric transformation?”

“Put images together in one coordinate system.”